

Topic 2.1

2.1: Functions in C

Functions are the fundamental mechanisms for providing software reuse. A function may be viewed as a black box that takes inputs from the invoking code and produces an output which is returned to the invoking code.

A simple example:

```
float midpoint( float x, float y)
{
    return (x + y) / 2;
}
```

The identifiers *x* and *y* are *formal* parameters, and allow the function to refer to its inputs, even though when the function is written the author does not know, in general, the actual inputs that will be supplied.

The *float* preceding the function name indicates the type of value returned by the function. The return statement causes the function to return to the function that invoked it, returning the specified value.

An invocation of the above function might look like this:

```
float a = 17.3, b=29.6, middle;
middle = midpoint(a,b);
```

In the invocation, *a* and *b* are *actual* parameters. These must match the formal parameters in number and type, on a position-by-position basis.

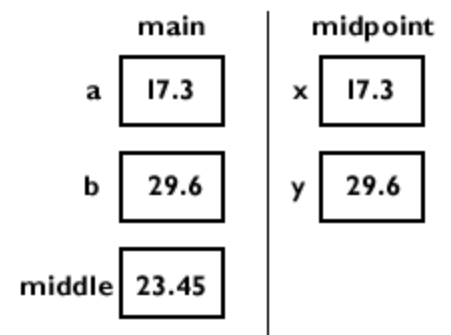
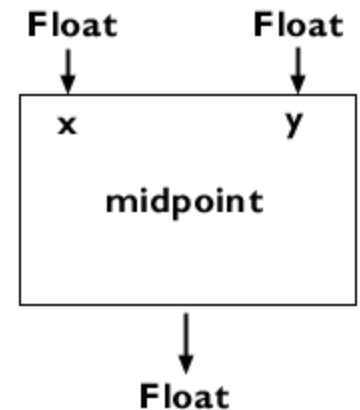
C passes all actual parameters by value: the value of each actual parameter is copied into the corresponding formal parameter. Changes made to the copy by the function are applied to the copy only; they do not affect the original actual parameter.

In call-by-reference, *which C does not have*, but which is available in most other languages and by default in Fortran, the *address* of an actual parameter is passed, rather than the *value* of the parameter.

Passing the address of the actual parameter allows the function to modify the value in a formal parameter, with the consequence of modifying the corresponding actual parameter.

C does not allow true call-by-reference, but it can be *faked* using pointers:

```
void triple_it( double *x )
{
```



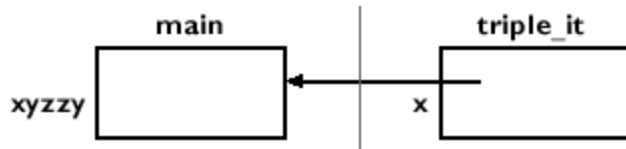
```

    *x = *x * 3;
}

double xyzzy=17.0;
triple_it(&xyzzy);

```

The ampersand (&) causes the address of variable xyzzy to be passed to the function, where it is used as a pointer.



Functions may contain local variable declarations. Declarations similar to the ones we have seen so far declare variables that are visible only to the enclosing function. The values of these variables are valid only so long as the function remains active. The function returns, and the values of such variables are lost.

The black box view may be distorted, as the function may not take any inputs, and/or may not produce any outputs.

Reference Parameters in Functions

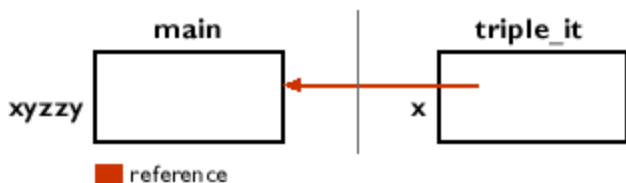
The necessity in C to use pointers to fake call-by-reference leads to an explosion of pointers, which in turn degrades the readability and understandability of the code. But in C++, we *can* use reference parameters to avoid this situation, resulting in a situation where we only use pointers when pointers are truly needed:

```

void triple_it( double &x )
{
    x = x * 3;
}

double xyzzy=17.0;
triple_it(xyzzy);

```



In this example, the programmer need not specify `&xyzzy` when `triple_it` is invoked. In fact, to do so would be erroneous. The compiler knows from the definition of `triple_it` that the first formal parameter is also a reference parameter and will automatically pass a reference to the first actual parameter in all invocations of the function.

A huge advantage of reference parameters is that the programmer only has to "get it right" in one place — the function definition/declaration. The compiler "does the right thing" everywhere the function is invoked.

Topic 2.2 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-2>)